Algorithm 1: `sign_basic(private_key, message, algorithm)`

函数作用: 使用私钥对消息进行基础数字签名，是实验一中“私钥签名”步骤的核心函数。

Input: `private_key`（RSA 私钥）；`message`（待签名字节串）；`algorithm`（签名算法，默认 RSA-PSS）

Output: 签名字节串

if `algorithm` 不是 `RSA-PSS` **then**

    抛出不支持算法异常

else

    使用 `RSA-PSS + SHA256` 对 `message` 进行签名

    return 签名字节串

Algorithm 2: `verify_basic(public_key, message, signature, algorithm)`

函数作用: 使用公钥验证基础签名，判断消息和签名是否来自匹配的私钥。

Input: `public_key`（RSA 公钥）；`message`（待验证消息）；`signature`（签名值）；
`algorithm`（签名算法）

Output: 验证通过返回 `True`，验证失败返回 `False`

if `algorithm` 不是 `RSA-PSS` **then**
    抛出不支持算法异常
else
    使用与签名阶段一致的 `RSA-PSS + SHA256` 参数验签
    if 验签过程未抛出异常 **then**
        return `True`
    else
        捕获 `InvalidSignature` 等异常
        return `False`

Algorithm 3: `sign_with_digest(private_key, message, hash_alg, sign_alg)`

函数作用: 先计算消息摘要，再对摘要值生成数字签名，用于实验二的完整性检测。

Input: `private_key`（RSA 私钥）；`message`（消息字节串）；`hash_alg`（摘要算法）；`sign_alg`（签名算法）

Output: 包含摘要、摘要展示值、签名值和算法标识的字典

if `sign_alg` 不是 `RSA-PSS` **then**

    抛出不支持算法异常

else

    digest ← 计算 `message` 的 `hash_alg` 摘要

    使用 `Prehashed(hash_alg)` 和 `RSA-PSS` 对摘要签名

    return 包含 `digest_hex`、`signature` 和展示片段的字典

Algorithm 4: `verify_with_digest(public_key, message, signature, hash_alg, sign_alg)`

函数作用: 接收方重新计算消息摘要，并使用公钥验证摘要签名是否有效。

Input: `public_key`（RSA 公钥）；`message`（收到的消息）；`signature`（摘要签名）；`hash_alg`；`sign_alg`

Output: 摘要签名验证结果

digest ← 重新计算 `message` 的摘要

if 使用 `RSA-PSS + Prehashed(hash_alg)` 验签成功 **then**

  return `True`

else

  return `False`

Algorithm 5: `sign_file(private_key, file_path, sign_path)`

函数作用: 对文件生成分离式数字签名，并将签名元信息写入独立签名文件。

Input: `private_key`（RSA 私钥）；`file_path`（原文件路径）；`sign_path`（签名文件路径）

Output: 无直接返回值；在 `sign_path` 写入 JSON 签名文件

digest ← 分块读取 `file_path` 并计算 SHA256 摘要

signature ← 使用 `RSA-PSS + Prehashed(SHA256)` 对摘要签名

构造包含格式标识、哈希算法、签名算法、时间戳和 `signature_b64` 的 JSON 包

将 JSON 包写入 `sign_path`

Algorithm 6: `verify_file_signature(public_key, file_path, sign_path)`

函数作用: 读取原文件和分离式签名文件，验证当前文件内容是否与签名匹配。

Input: `public_key`（RSA 公钥）；`file_path`（待验证文件）；`sign_path`（签名文件）

Output: 文件签名验证结果

读取 `sign_path` 中的算法字段和 `signature_b64`

digest ← 重新计算当前 `file_path` 的摘要

if 公钥对文件摘要验签成功 **then**

    return `True`

else

    捕获签名异常、文件异常或 JSON 解析异常

    return `False`

Algorithm 7: `sign_with_metadata(private_key, message, metadata)`

函数作用: 将消息内容和元数据整体绑定到签名中，生成带上下文保护的签名数据包。

Input: `private_key`（RSA 私钥）；`message`（消息字节串）；`metadata`（发送者、接收者、算法、时间戳和随机数等字段）

Output: 包含消息、元数据、算法标识和签名值的签名包

将 `message` 转为 Base64，并与 `metadata` 组成待签名对象

使用 canonical JSON 对待签名对象进行确定性序列化

signature ← 使用 `RSA-PSS + SHA256` 对序列化结果签名

return 包含 `message_b64`、`metadata` 和 `signature_b64` 的字典

Algorithm 8: `verify_metadata_signature(public_key, package)`

函数作用: 解析带元数据的签名包，检查元数据格式并重新序列化后验签。

Input: `public_key`（RSA 公钥）；`package`（带元数据签名包）

Output: 元数据签名验证结果

读取 `package` 中的 `metadata`、`message_b64` 和 `signature_b64`

if 元数据字段缺失或时间戳、随机数格式错误 **then**

    return `False`

else

    按签名阶段相同规则重新序列化消息和元数据

    使用公钥验证 `signature_b64`

    return 验签结果

Algorithm 9: `sign_then_encrypt(sender_private_key, receiver_public_key, plaintext, metadata)`

函数作用: 先对明文和元数据签名，再把明文、签名和元数据整体加密给接收方。

Input: `sender_private_key`（发送方私钥）；`receiver_public_key`（接收方公钥）；
`plaintext`；`metadata`

Output: 外层安全消息包

构造包含明文和元数据的待签名对象

signature ← 使用发送方私钥签名该对象

构造内部明文包：明文、元数据、签名和算法标识

随机生成 AES 会话密钥，并用 `AES-256-GCM` 加密内部包

使用接收方公钥通过 `RSA-OAEP` 加密会话密钥

return 包含 `encrypted_key_b64`、`nonce_b64`、`ciphertext_b64` 和 `tag_b64` 的安全包

Algorithm 10: decrypt_then_verify(receiver_private_key, sender_public_key, secure_package)

函数作用: 先解密“先签名后加密”消息包，再对内部签名进行验证。

Input: receiver_private_key（接收方私钥）；sender_public_key（发送方公钥）；
secure_package（安全消息包）

Output: 包含解密状态、验签状态、明文和错误原因的字典

使用接收方私钥解密 encrypted_key_b64 得到会话密钥

使用会话密钥和 AES-GCM 解密内部明文包，并校验 tag

解析内部包中的明文、元数据和签名

使用发送方公钥验证内部签名

return decrypt_success、verify_success、plaintext 和 error

Algorithm 11: encrypt_then_sign(sender_private_key, receiver_public_key, plaintext, metadata)

函数作用: 先加密明文，再对密文包及关键元数据生成外层数字签名。

Input: sender_private_key（发送方私钥）；receiver_public_key（接收方公钥）；plaintext；metadata

Output: 带外层签名的密文包

随机生成 AES 会话密钥，并使用 AES-GCM 加密 plaintext

使用接收方公钥加密 AES 会话密钥

构造包含算法套件、加密会话密钥、随机数、密文、标签和元数据的外层包

使用发送方私钥对外层关键字段执行 RSA-PSS 签名

return 带 signature_b64 的密文包

Algorithm 12: `verify_then_decrypt(receiver_private_key, sender_public_key, signed_cipher_package)`

函数作用: 先验证外层签名，确认密文包来源与完整性，再尝试解密密文。

Input: `receiver_private_key`（接收方私钥）；`sender_public_key`（发送方公钥）；`signed_cipher_package`（带签名密文包）

Output: 包含验签状态、解密状态、明文和错误原因的字典

提取外层关键字段并使用发送方公钥验证 `signature_b64`

if 外层验签失败 **then**

    return 验签失败且不执行解密

else

    使用接收方私钥解密会话密钥

    使用会话密钥解密密文并校验 `tag`

    return 验签和解密结果

Algorithm 13: `build_secure_envelope(sender_private_key, receiver_public_key, plaintext, sender_id, receiver_id)`

函数作用: 构造“数字信封 + 数字签名”综合安全包，同时提供保密性、完整性和身份认证。

Input: `sender_private_key`；`receiver_public_key`；`plaintext`；`sender_id`；`receiver_id`

Output: 包含加密会话密钥、密文、完整性标签、身份字段和外层签名的安全信封

使用 `AES-GCM` 加密明文，得到 `ciphertext`、`nonce` 和 `tag`

使用接收方公钥通过 `RSA-OAEP` 加密 AES 会话密钥

构造包含格式标识、算法套件、发送者、接收者、时间戳和密文字段的安全信封

使用发送方私钥对安全信封关键字段签名

return 带 `signature_b64` 的安全信封字典

Algorithm 14: open_secure_envelope(receiver_private_key, sender_public_key, secure_envelope)

函数作用: 打开数字信封：先验证外层签名，再解密会话密钥和密文，并校验完整性标签。

Input: receiver_private_key（接收方私钥）；sender_public_key（发送方公钥）；secure_envelope（安全信封）

Output: 签名验证结果、解密结果、完整性结果、明文和错误原因

验证安全信封外层签名及算法套件是否符合预期

if 签名验证失败 **then**

  return signature_valid=False 且不继续解密

else

  使用接收方私钥解密会话密钥

  使用 AES 会话密钥解密密文，并由 AES-GCM 校验 tag

  return signature_valid、decrypt_success、integrity_valid 和明文

Algorithm 15: `run_all_experiments()`

函数作用: 主程序实验入口，统一生成密钥、准备消息、执行七个实验并汇总结果。

Input: 无显式参数；函数内部准备实验消息、文件和密钥

Output: 包含参与方信息、消息信息和七个实验结果的总结果字典

生成 Alice、Bob 以及错误测试用 RSA 密钥对

准备原始消息、篡改消息和二进制消息

依次执行基础签名、摘要签名、文件签名、元数据签名、先签名后加密、先加密后签名和数字信封实验

记录正常流程、篡改测试和错误密钥测试结果

写入生成时间并返回 `results` 字典

Algorithm 16: `print_result_summary(results)`

函数作用: 按实验编号输出结果摘要，用于终端展示和实验报告填写。

Input: `results`（`run_all_experiments()` 返回的总结果字典）

Output: 终端摘要输出；不返回新数据

打印参与方、公钥指纹和实验消息摘要

按实验一到实验七的顺序读取关键结果字段

将布尔结果转换为“通过/失败”或“成功/失败”文本

打印签名、摘要、密文和标签的必要展示片段

避免输出私钥和完整敏感数据